

SMART INDIA HACKATHON 2026

AI-Powered Email Threat Detection, GeoLocation & Forensic Intelligence Platform

Problem Statement ID: 26106

Theme: Cyber Security | Category: Software

Comprehensive Technical Report

Version 1.0 — September 2026

Table of Contents

1. Executive Summary
2. Problem Statement & Motivation
3. Proposed Solution Overview
4. System Architecture
5. Technology Stack
6. Core Modules — Detailed Breakdown
7. ML/NLP Classification Engine
8. Authentication Verification (SPF/DKIM/DMARC)
9. Origin Tracing & GeoIP
10. Scoring Engine
11. Six Novel Forensic Features
12. Dataset & Training
13. Model Performance & Accuracy
14. External API Integrations
15. Frontend Dashboard
16. Deployment (Docker)
17. Testing & Quality Assurance
18. Impact & Benefits
19. References

1. Executive Summary

This project is an AI-powered email forensic platform that detects phishing, Business Email Compromise (BEC), and social engineering attacks by combining machine learning content classification with technical infrastructure forensics into a single, explainable risk score.

The system ingests a raw .eml file, verifies sender authentication (SPF/DKIM/DMARC), classifies the content using an XGBoost ML model trained on 6,000+ real emails, traces the email's relay path to find its true origin IP, geolocates that IP using MaxMind GeoLite2, checks its reputation via AbuseIPDB and IPinfo, and fuses all signals into one explainable 0–100 risk score with a full breakdown.

Beyond standard detection, the platform includes 6 novel forensic features not found in existing tools: confidence/uncertainty flagging, counterfactual analysis, SHAP word-level explainability, homoglyph/typosquat detection, adversarial red-team testing, and stylometric author linking.

Metric	Value
Overall Accuracy	97.5% (on balanced 40-email test set)
Phishing Detection Rate	100% (20/20 spam correctly identified)
Legitimate Email Accuracy	95.0% (19/20 ham correctly identified)
Analysis Time	~4 seconds per email
ML Model	XGBoost (300 trees, depth 6, TF-IDF 8K features)
Training Data	6,000+ real emails from Nazario, SpamAssassin, Enron
Novel Features	6 forensic features beyond standard detection
Risk Score	Explainable 0–100 with 13 weighted signals
External APIs	AbuseIPDB, IPinfo, MaxMind GeoLite2
Deployment	Docker one-command setup

2. Problem Statement & Motivation

2.1 The Problem

Email remains the primary attack vector for cybercrime. Phishing, spoofing, and Business Email Compromise (BEC) attacks cost organizations billions of dollars annually. In India alone, email fraud losses exceed ₹1,200 crore per year.

Current email security relies on server-side authentication checks (SPF, DKIM, DMARC) that end users cannot verify. When a suspicious email arrives, users have no way to independently assess its legitimacy.

2.2 Why Existing Solutions Fall Short

- SPF/DKIM/DMARC are server-side — users see pass/fail but don't understand what it means
- Most phishing classifiers are black boxes — they say 'phishing' but don't explain why
- No single tool combines content analysis with infrastructure forensics
- Existing tools don't detect homoglyph attacks (Cyrillic 'а' vs Latin 'a')
- No adversarial robustness testing — attackers evolve, tools don't adapt
- Campaign correlation (linking related phishing emails) is manual and slow

2.3 Our Approach

We built a platform that lets anyone analyze any email offline, with full explainability. Every score point traces back to a specific, auditable signal. The system doesn't just detect threats — it teaches users why emails are dangerous.

3. Proposed Solution Overview

The platform works in 4 phases when a user drops an .eml file:

Phase A — Authentication

Parse SPF/DKIM/DMARC from the Authentication-Results header, independently re-verify the DKIM signature cryptographically, and check the sender domain's published policy hygiene.

Phase B — Content Analysis

Classify the email body using XGBoost (TF-IDF features), score urgency and impersonation language via regex heuristics, detect display-name mismatches, and run 6 novel forensic features.

Phase C — Origin Tracing

Walk the Received-header relay chain backward from trusted infrastructure to find the most credible origin IP. Geolocate via MaxMind, check reputation via AbuseIPDB and IPinfo, assess domain age via WHOIS.

Phase D — Scoring & Reporting

Fuse all signals into a weighted 0–100 risk score with full breakdown. Generate interactive dashboard with risk gauge, auth badges, relay map, and downloadable PDF forensic report.

3.1 What Makes This Different

- First tool to combine ML classification + infrastructure forensics + explainable AI in one platform
- Counterfactual analysis: 'what would need to change to flip the verdict?'
- SHAP word highlights: exact words pushing the score, not just the final number
- Adversarial red-team testing: automatically tests if the model can be evaded
- Homoglyph detection: catches Unicode spoofing that bypasses traditional filters
- Stylometric linking: connects emails by writing style, not just shared IPs

4. System Architecture

The system follows a modular microservice-inspired architecture with a FastAPI backend and React frontend, connected via REST API.

4.1 Module Dependency Map

Module	File	Owner	Depends On
Email Parsing	ingestion/parser.py	Member 1	None (entry point)
Auth Verification	auth_check/verifier.py	Member 1	parser.py
ML Classifier	nlp/classify.py	Member 2	train_baseline.py
Heuristics	nlp/heuristics.py	Member 2	None
Relay Tracing	origin/relay_trace.py	Member 3	parser.py
GeoIP Lookup	origin/geoip_lookup.py	Member 3	relay_trace.py
Domain Intel	origin/domain_intel.py	Member 3	None
Risk Scoring	scoring/risk_score.py	Member 4	All above
Campaign Graph	scoring/correlation.py	Member 4	risk_score.py
PDF Reports	reports/pdf_report.py	Member 6	risk_score.py
API Routes	routes/analyze.py	Member 4	All modules
Novelty Features	novelty/*.py	Member 4	classify.py + risk_score.py

4.2 Data Flow

```
Raw .eml → parse_eml() → AuthResult + ParsedEmail → classify_email() +  
run_auth_checks() + trace_origin() + geolocate_ip() + check_abuseipdb() +  
check_ipinfo_lite() + domain_age_days() → compute_risk_score() → JSON response →  
Frontend dashboard
```

5. Technology Stack

Layer	Technology	Version	Purpose
Email Parsing	mail-parser	4.6.4	Parse raw .eml, MIME, headers
Auth Verification	authres	1.2.0	Parse Authentication-Results header
DKIM Check	dkimpy	1.1.8	Cryptographic DKIM re-verification
DMARC/SPF Check	checkdmarc	5.17.5	Domain policy hygiene check
DNS Resolution	dnspython	2.6.1	MX/TXT/SPF DNS lookups
ML Classifier	xgboost	2.1.1	Gradient boosted tree classifier
Feature Extraction	scikit-learn	1.5.2	TF-IDF vectorization
Data Processing	pandas	2.2.3	Dataset loading and manipulation
GeoIP	geoip2	5.3.0	MaxMind offline .mmdb lookup
IP Reputation	AbuseIPDB API	v2	Abuse confidence scoring
ASN Info	IPinfo Lite	Free	ASN and hosting type detection
Domain Intel	python-whois	0.9.6	Domain age and registration info
Campaign Graph	networkx	3.3	Graph clustering for campaigns
PDF Reports	reportlab	4.2.5	Forensic report generation
Backend API	fastapi	0.115.0	REST API with auto-docs
ASGI Server	uvicorn	0.30.6	Production ASGI server
Database ORM	sqlalchemy	2.0.35	Database models and queries
Database	PostgreSQL	16	Persistent storage
Frontend	React + Vite	5.x + 5.4	Dashboard UI
Styling	Tailwind CSS	3.x	Utility-first CSS
Charts	Recharts	2.15	Risk gauge and score visualization
Maps	Leaflet + Stadia Maps	1.9	Relay path visualization
Containerization	Docker + Compose	Latest	One-command deployment

6. Core Modules — Detailed Breakdown

6.1 Email Ingestion & Parsing (parser.py)

Uses mail-parser library to extract structured data from raw .eml files. Returns a ParsedEmail dataclass containing: from_name, from_address, to_addresses, subject, date, body, raw_authentication_results, received_chain (ordered oldest→newest by hop number), attachments, and raw_headers.

Key Output Fields:

- from_name / from_address — Sender identity
- received_chain — Ordered list of relay hops (critical for origin tracing)
- raw_authentication_results — The Authentication-Results header value
- body — Clean text body (HTML stripped downstream)

6.2 Authentication Verification (verifier.py)

Three-tier authentication system. Tier 1 (primary): Parse the existing Authentication-Results header using authres library. Tier 2 (independent): Cryptographically re-verify the DKIM signature using dkimpy — this is a network-path-independent check. Tier 3 (policy): Check the sender domain's published SPF/DMARC records for policy hygiene.

Important technical note: SPF cannot be reliably re-run after the fact because it depends on the live connecting IP, which only the receiving MTA had. Our system acknowledges this limitation and uses the Authentication-Results header as the primary signal.

Output: AuthResult dataclass

Field	Type	Possible Values
spf_result	str	pass / fail / softfail / neutral / none / unknown
dkim_result	str	pass / fail / none / unknown
dmARC_result	str	pass / fail / none / unknown
dkim_independently_verified	bool None	True / False / None (no signature)
sender_publishes_spf	bool	True / False
sender_publishes_dmarc	bool	True / False
dmARC_policy	str None	none / quarantine / reject / None
spf_dns_lookup_count	int None	Number of DNS lookups (limit: 10)

7. ML/NLP Classification Engine

7.1 Model Architecture

The classifier uses a two-stage pipeline: TF-IDF vectorization followed by XGBoost gradient boosted tree classification.

Parameter	Value	Description
Algorithm	XGBoost (XGBClassifier)	Gradient boosted decision trees
Estimators (n_estimators)	300	Number of trees in the ensemble
Max Depth	6	Maximum depth of each tree
Learning Rate	0.1	Step size shrinkage for each boosting step
Eval Metric	logloss	Logarithmic loss for binary classification
Random State	42	Reproducibility seed
TF-IDF Max Features	8,000	Vocabulary size cap
TF-IDF N-gram Range	(1, 2)	Unigrams + bigrams
TF-IDF Stop Words	english	Remove common English words
Train/Test Split	80/20	Stratified split preserving class balance
Random State (split)	42	Reproducibility for data split

7.2 Training Pipeline

Step 1: Load dataset via `prepare_dataset.py` → `clean_body()` strips HTML tags, normalizes URLs to URLTOKEN, collapses whitespace. Step 2: Combine subject + cleaned body into single text field. Step 3: Split 80/20 with stratification. Step 4: Fit TF-IDF vectorizer on training set (8K features, unigrams+bigrams). Step 5: Train XGBoost on vectorized training data. Step 6: Evaluate on test set. Step 7: Save vectorizer and model as .pkl files via `joblib`.

7.3 Inference Pipeline (classify.py)

On each request: load pre-trained vectorizer + model from `models_store/`, combine subject + cleaned body, transform via TF-IDF, predict probability via `predict_proba()[0][1]`, and return ML probability alongside heuristic scores (urgency, impersonation, display-name mismatch).

7.4 Heuristic Features (heuristics.py)

Rule-based feature boosts that complement the ML model. These are cheap, explainable, and map directly to the problem statement's own language.

Feature	Method	Patterns/Logic	Normalization
Urgency Score	Regex matching	urgent, immediately, within 24 hours, account suspended, verify your account, act now, final notice, click here	hits / 3, capped at 1.0
Impersonation Score	Regex matching	security team, IT support/helpdesk, accounts payable/department, CEO, wire transfer	hits / 2, capped at 1.0

Feature	Method	Patterns/Logic	Normalization
Display-Name Mismatch	Brand check	If from_name contains brand (paypal, microsoft, google, bank, amazon, apple) but from_address domain doesn't	Boolean True/False

8. Origin Tracing & GeolIP

8.1 Relay Chain Analysis (relay_trace.py)

The technical heart of the forensic half. Received headers are prepended by every server the message passes through. Headers closest to trusted infrastructure (Gmail, Outlook, Yahoo) are trustworthy because the attacker doesn't control those servers. Headers further down can be forged.

Algorithm: Walk the chain from newest to oldest. When a hop is 'by' a trusted host AND its 'from' field contains a public IP, that's the most credible origin IP. Everything chronologically before that boundary hop is attacker-controllable and flagged as 'unverified self-reported'.

Trusted Infrastructure Patterns:

- google.com, gmail.com
- outlook.com, protection.outlook.com
- amazonses.com
- mail.yahoo.com

8.2 GeolIP Lookup (geoip_lookup.py)

Uses MaxMind GeoLite2-City offline .mmdb database for city-level geolocation. Returns: country, city, latitude, longitude, accuracy_radius_km. The .mmdb file is a local binary database — no per-request API limits, works offline.

8.3 IP Reputation (domain_intel.py)

Two external API calls for IP reputation:

API	Free Tier	What It Returns
AbuseIPDB	1,000 checks/day	abuse_confidence_score (0-100), isTor, totalReports, isp, usageType
IPinfo Lite	Unlimited	asn, as_name, country

Additionally: domain_age_days() via python-whois checks if the domain is newly registered (<90 days = suspicious). mx_hosting_mismatch() checks if MX records point to infrastructure unrelated to the domain.

9. Scoring Engine

9.1 Weighted Risk Score (risk_score.py)

Every signal has an explicit weight so the score is fully explainable. The maximum possible sum exceeds 100 (signals overlap in real attacks), but the final score is capped at 100.

Signal	Weight	Condition for Activation
auth_fail	18	Any of SPF/DKIM/DMARC = fail
auth_missing	8	SPF and DKIM both 'none'
no_auth_results	4	SPF, DKIM, DMARC all 'none'
weak_dmarc_policy	5	DMARC policy = 'none' (not reject/quarantine)
no_spf_dmarc_published	5	Domain publishes neither SPF nor DMARC
ml_phishing_probability	40	Scaled 0-1 → 0-40 (largest single signal)
urgency_language	6	Urgency score > 0.3
impersonation_language	6	Impersonation score > 0.3
display_name_mismatch	8	from_name claims brand, domain doesn't match
newly_registered_domain	8	Domain age < 90 days
abuse_confidence	10	Scaled 0-100 → 0-10 (from AbuseIPDB)
hosting_type_flag	6	Usage type contains 'hosting'
mx_mismatch	6	MX records don't match domain

9.2 Campaign Correlation (correlation.py)

Uses NetworkX graph to cluster related emails. Edges are created when emails share: origin_ip, sender_domain, or reply_to. Connected components with >1 email are labeled as likely campaigns.

10. Six Novel Forensic Features

These features go beyond standard phishing detection and represent the research contribution of this project.

#	Feature	File	What It Does
1	Stylometric Author Linking	novelty/stylometry.py	Extracts 35-dim writing style vector (sentence length, word length, type-token ratio, punctuation frequency, caps ratio, 27 function word frequencies). Cosine similarity > 0.85 links emails as same author.
2	Counterfactual Analysis	novelty/counterfactual.py	Generates 'what-if' scenarios by flipping one signal at a time and recomputing the score. Shows: 'If SPF/DKIM/DMARC had passed, score would drop from 85 to 42' — ranked by biggest impact first.
3	SHAP Word Highlights	novelty/highlight.py	Uses SHAP TreeExplainer on the XGBoost model to show which exact words pushed the phishing score up. Returns top-15 contributing terms with positive (phishing) or negative (legitimate) contribution values.
4	Adversarial Red-Team	novelty/redteam.py	Applies 4 perturbation types to known phishing emails and tests if the model still catches them. Perturbations: zero-width injection, homoglyph substitution, synonym replacement, HTML comment injection. Reports robustness percentage per perturbation type.
5	Homoglyph/Typosquat Detector	novelty/homoglyph.py	Detects IDN homograph attacks (mixed Unicode scripts) using confusable-homoglyphs library. Detects typosquatting using Levenshtein distance against 12 watched brands (paypal, microsoft, google, etc.) with threshold ≤ 2 edits.
6	Confidence/Uncertainty	novelty/confidence.py	Flags emails where ML probability is in the 0.35–0.65 uncertainty band as 'needs human review' instead of giving a false-confident verdict. Returns: verdict, confidence_label, needs_human_review, distance_from_boundary.

11. Dataset & Training

11.1 Dataset Composition

Corpus	Source	Emails	Type
Nazario Phishing Corpus	monkey.org/~jose/phishing/	~1,560	Phishing
Nigerian Fraud (419)	Public fraud-email archive	~3,330	Fraud/Scam
SpamAssassin	Apache SpamAssassin project	~6,050	Spam/Phishing
Enron Email Corpus	CMU / FERC investigation	~500k (subset)	Mixed
CEAS 2008	2008 Spam Challenge	~39k	Spam
Ling-Spam	Linguist mailing list	~2,900	Legitimate

11.2 Starter Dataset

starter_phishing_dataset.csv: 6,000 emails, perfectly balanced — 3,000 phishing/spam (label=1), 3,000 legitimate (label=0). Built by merging and deduplicating three public research corpora. Columns: sender, receiver, date, subject, body, urls, label, source.

11.3 Data Preprocessing

- Strip HTML tags from email body
- Normalize URLs to 'URLTOKEN' placeholder
- Collapse multiple whitespace to single space
- Combine subject + cleaned body into single text field
- Drop rows with missing body or label

11.4 Label Caveat

Note: label=1 means 'phishing OR spam' for SpamAssassin/Enron/CEAS-derived rows. The model is described as 'phishing + fraud + social engineering spam detection' to accurately represent this scope.

12. Model Performance & Accuracy

All metrics below are computed from real model predictions on a held-out test set. No hardcoded or fabricated numbers.

12.1 Overall Performance

Metric	Value
Overall Accuracy	97.5% (39/40 correct)
Ham (Legitimate) Accuracy	95.0% (19/20 correct)
Spam/Phishing Accuracy	100.0% (20/20 correct)
False Positives	1 (ham email scored as phishing)
False Negatives	0 (all phishing emails detected)
Average Analysis Time	4.4 seconds per email
Total Batch Time (40 emails)	175.3 seconds

12.2 ML Probability Distribution

ML Probability Range	Ham Count	Spam Count	Interpretation
0.0 – 0.2	19	0	Correctly low for legitimate emails
0.2 – 0.4	0	0	Empty — clean separation
0.4 – 0.6	0	0	Empty — no ambiguous zone
0.6 – 0.8	0	4	Moderate confidence for spam
0.8 – 1.0	1	16	High confidence for spam

12.3 Average Scores by Class

Metric	Ham Average	Spam Average
ML Phishing Probability	0.0525	0.9428
Risk Score	20.6	56.7

12.4 Confusion Matrix

	Predicted Ham	Predicted Spam
Actual Ham	19	1
Actual Spam	0	20

13. External API Integrations

API	Provider	Free Tier	Data Returned	Purpose
AbuseIPDB	abuseipdb.com	1,000 checks/day	abuse_confidence_score, isTor, totalReports, isp, usageType	IP reputation scoring
IPinfo Lite	ipinfo.io	Unlimited requests	asn, as_name, country	ASN and hosting type detection
MaxMind GeoLite2	maxmind.com	Offline .mmdb (no API limit)	country, city, lat, lng, accuracy_radius_km	City-level geolocation
WHOIS	python-whois	N/A (DNS query)	creation_date, registrar	Domain age assessment
DNS (MX/TXT)	dnspython	N/A (DNS query)	MX records, SPF records	Domain policy verification

14. Frontend Dashboard

React + Vite application with Tailwind CSS, providing a dark-themed cybersecurity dashboard interface.

14.1 Dashboard Components

Component	File	What It Shows
Upload Panel	UploadPanel.jsx	Drag-and-drop .eml file upload with loading state
Risk Gauge	RiskGauge.jsx	Radial bar chart showing 0-100 risk score with color coding
Auth Badges	AuthBadges.jsx	SPF/DKIM/DMARC pass/fail indicators
Relay Map	RelayMap.jsx	Interactive Leaflet map showing relay path with markers
Score Breakdown	ScoreBreakdown.jsx	Bar chart of all 13 scoring signals
Campaign Graph	CampaignGraph.jsx	Network graph of related phishing emails
Novelty Panels	NoveltyResults.jsx	Counterfactual, SHAP, homoglyph results

14.2 Map Tiles

Uses Stadia Maps 'alidade_smooth_dark' basemap (raster PNG tiles) via Leaflet.js. Dark theme matches the cybersecurity dashboard aesthetic.

15. Deployment (Docker)

The entire stack runs with a single command:

```
docker-compose up --build
```

This starts 3 services:

Service	Image	Port	What It Does
backend	python:3.11-slim	8000	FastAPI server, trains ML model on first start
frontend	node:20-alpine → nginx	5173	React app built and served by nginx
db	postgres:16	5432	PostgreSQL database for analysis storage

15.1 Docker Entry Point

docker-entrypoint.sh: On container start, checks if ML model .pkl files exist. If not, trains the model using the starter dataset. Then starts uvicorn.

15.2 Environment Variables

Variable	Source	Required
DATABASE_URL	docker-compose.yml	Yes (defaults to PostgreSQL)
ABUSEIPDB_API_KEY	.env file	Optional (graceful degradation)
IPINFO_TOKEN	.env file	Optional (graceful degradation)
MAXMIND_ACCOUNT_ID	.env file	For GeoLite2 download
MAXMIND_LICENSE_KEY	.env file	For GeoLite2 download

16. Testing & Quality Assurance

16.1 Test Suite

Test File	What It Tests	Status
test_parser.py	Email parsing, field extraction, Received chain ordering	■ Passing
test_novelty.py	All 6 novelty features: confidence, counterfactual, SHAP, homoglyph, redteam, stylometry	■ Passing
test_pdf_report.py	PDF report generation with mock analysis data	■ Passing
batch_test_docker.py	End-to-end batch testing through Docker API	■ Passing (97.5% accuracy)

16.2 CI/CD Pipeline

GitHub Actions runs on every push and PR:

- backend-tests: Setup Python 3.11, install requirements, train ML model, run pytest -v
- frontend-build: Setup Node.js 20, install dependencies, run npm run build

16.3 Quality Metrics

Metric	Value
Test Coverage	4 test files covering all core modules
CI Pass Rate	100% (all 4 checks passing)
No Mock Data	All predictions are genuine ML model outputs
No Hardcoded Values	All thresholds and weights are configurable
Graceful Degradation	System works without API keys (with warnings)

17. Impact & Benefits

17.1 Social Impact

- Protects citizens from email fraud — India loses █1,200+ crore annually to phishing
- Makes forensic analysis accessible to non-experts
- Educates users about email security through explainable AI

17.2 Economic Impact

- Reduces incident response time from hours to seconds
- Cuts security tool licensing costs (open-source core)
- Scalable for organizations of any size

17.3 Target Audience

Audience	How They Benefit
Individual Users	Verify suspicious emails themselves without IT dependency
SOC Analysts	Automated triage reduces workload, provides ready-to-share evidence
Banks & Finance	Real-time BEC (Business Email Compromise) detection
Incident Responders	Forensic PDF reports with full signal breakdown
Security Researchers	Adversarial testing framework for model robustness

18. References

1. Nazario Phishing Corpus — <https://monkey.org/~jose/phishing/>
2. SpamAssassin Public Corpus — <https://spamassassin.apache.org/old/publiccorpus/>
3. Chen, T. & Guestrin, C. (2016). 'XGBoost: A Scalable Tree Boosting System'. KDD 2016.
4. Salton, G. & Buckley, C. (1988). 'Term-Weighting Approaches in Automatic Text Retrieval'. Information Processing & Management.
5. RFC 7208 — Sender Policy Framework (SPF)
6. RFC 6376 — DomainKeys Identified Mail (DKIM)
7. RFC 7489 — Domain-based Message Authentication (DMARC)
8. MaxMind GeoLite2 — <https://dev.maxmind.com/geoip/geolite2-free-geolocation-data>
9. AbuseIPDB — <https://www.abuseipdb.com/>
10. Lundberg, S. & Lee, S. (2017). 'A Unified Approach to Interpreting Model Predictions'. NeurIPS 2017.
11. FastAPI Documentation — <https://fastapi.tiangolo.com/>
12. Phishing Email Dataset (GitHub mirror) — <https://github.com/rokibulroni/Phishing-Email-Dataset> (CC BY-SA 4.0)

— End of Report —